

BasicCNN 网络结构与代码设计说明

本文档用于说明本项目中 BasicCNN 图像分类流程的设计思路，便于后续撰写实验报告。

1. 任务目标

本项目使用 STL-10 数据集完成 10 类图像分类任务。数据集中的图像为 RGB 彩色图像，尺寸为 96 x 96，类别包括：

airplane, bird, car, cat, deer, dog, horse, monkey, ship, truck

模型输入为一张 STL-10 图像，输出为 10 个类别对应的 logits，训练时使用交叉熵损失函数进行监督学习。

2. 整体代码结构

项目将数据读取、模型定义、训练评估和命令行入口拆分到不同文件中，便于维护和实验扩展。

```
src/
|-- config.py      # 统一管理路径、类别、训练超参数
|-- dataset.py     # 读取 STL10 文件夹数据，构造 train/valid/test DataLoader
|-- models.py      # 定义 BasicCNN 网络结构
|-- train.py       # 训练、验证、测试评估、保存模型
main.py            # 命令行入口
```

这种设计的好处是：模型结构和训练流程相互独立，后续如果要加入 ImprovedCNN、ResNet 或更多实验，只需要在 models.py 中增加模型，并在训练入口中选择即可。

3. 数据读取与预处理设计

数据集按如下目录组织：

```
STL10/  
|-- train/  
|   |-- airplane/  
|   |-- bird/  
|   `-- ...  
`-- test/  
    |-- airplane/  
    |-- bird/  
    `-- ...
```

`dataset.py` 中实现了 `STL10FolderDataset`，它会按照固定类别顺序遍历每个类别文件夹，并为每张图片分配对应标签。固定类别顺序可以保证训练、验证和测试阶段的标签含义一致。

训练集会从 `STL10/train` 中划分出一部分作为验证集，默认验证集比例为 `0.15`。代码中分别构造了训练集和验证集两个 `Dataset` 对象，避免训练数据增强影响验证集。

训练阶段使用的数据增强包括：

```
RandomCrop(96, padding=8)  
RandomHorizontalFlip()  
ToTensor()  
Normalize(mean, std)
```

其中 `RandomCrop` 可以增强模型对物体位置变化的鲁棒性，`RandomHorizontalFlip` 可以扩充图像姿态变化，降低过拟合风险。验证集和测试集不使用随机增强，只进行尺寸统一和标准化，以保证评估结果稳定。

4. BasicCNN 网络结构

BasicCNN 是一个针对 STL-10 的轻量卷积神经网络。由于 STL-10 图像尺寸为 `96 x 96`，网络通过多层卷积提取局部纹理、边缘、形状等特征，并通过池化逐步降低空间分辨率。

整体结构如下：

Input: 3 x 96 x 96

ConvBlock 1: 3 -> 32 96 x 96 -> 48 x 48

ConvBlock 2: 32 -> 64 48 x 48 -> 24 x 24

ConvBlock 3: 64 -> 128 24 x 24 -> 12 x 12

ConvBlock 4: 128 -> 256 12 x 12 -> 6 x 6

AdaptiveAvgPool2d: 256 x 6 x 6 -> 256 x 1 x 1

Flatten: 256

Linear: 256 -> 128

ReLU

Dropout

Linear: 128 -> 10

每个 ConvBlock 内部结构为：

Conv2d

BatchNorm2d

ReLU

Conv2d

BatchNorm2d

ReLU

MaxPool2d

Dropout2d

4.1 卷积层设计

卷积核大小使用 3×3 ，padding 设置为 1，这样卷积操作本身不会改变特征图尺寸。 3×3 卷积是图像分类中常用的基础设计，既能提取局部空间特征，又不会引入过多参数。

每个卷积块中连续使用两层卷积，可以在同一尺度下增强特征表达能力。随着网络加深，通道数从 32 逐步增加到 256，使模型能够从低级纹理逐渐学习到更高级的语义特征。

4.2 Batch Normalization

每个卷积层后使用 BatchNorm2d。它可以稳定中间特征分布，使训练过程更平稳，同时允许模型使用相对较大的学习率。对于从零训练的 CNN，BatchNorm 通常能改善收敛速度和最终效果。

4.3 ReLU 激活函数

模型采用 ReLU 作为非线性激活函数。ReLU 计算简单，能够缓解梯度消失问题，是 CNN 中常用的基础激活函数。

4.4 Max Pooling

每个卷积块末尾使用 `MaxPool2d(kernel_size=2)`，将特征图长宽减半。对于 96×96 输入，经过 4 次池化后空间尺寸变为 6×6 。这种设计可以逐步扩大感受野，同时降低计算量。

4.5 Dropout

卷积块中使用 `Dropout2d`，分类器中使用普通 `Dropout`。`Dropout` 可以随机屏蔽部分特征，减少模型对训练样本的记忆，提高泛化能力。

4.6 Adaptive Average Pooling

分类头前使用 `AdaptiveAvgPool2d((1, 1))`，将 $256 \times 6 \times 6$ 的特征图压缩为 $256 \times 1 \times 1$ 。这种设计减少了全连接层参数量，也让模型对输入特征图尺寸更稳定。

5. 训练流程设计

训练入口在 `main.py` 中，通过命令行参数控制训练或评估：

```
python main.py --mode train --model basic
python main.py --mode eval --model basic
```

训练流程主要包括：

1. 设置随机种子，保证实验可复现
2. 构造 `train/valid/test DataLoader`
3. 创建 `BasicCNN` 模型
4. 使用 `CrossEntropyLoss` 作为分类损失
5. 使用 `AdamW` 优化器更新参数
6. 使用 `CosineAnnealingLR` 调整学习率
7. 每个 `epoch` 后在验证集上评估
8. 保存验证集准确率最高的模型
9. 将训练曲线数据保存到 `outputs/logs/basic_history.csv`

优化器选择 AdamW ，它相比普通 Adam 更好地处理权重衰减，常用于深度学习模型训练。学习率调度器使用 CosineAnnealingLR ，使学习率在训练过程中逐渐减小，有助于后期模型收敛。

6. 评估指标

测试阶段会输出以下指标：

```
Loss
Accuracy
Macro Precision
Macro Recall
Macro F1-score
Confusion Matrix
```

其中 Accuracy 反映整体分类正确率，Precision/Recall/F1 使用 macro 平均，可以更均衡地观察各类别表现，避免只看整体准确率时忽略类别间差异。

混淆矩阵的行表示真实类别，列表示预测类别。通过混淆矩阵可以分析哪些类别容易被模型混淆，例如动物类之间、交通工具类之间是否存在误判。

7. 当前 BasicCNN 的定位

BasicCNN 的目标不是追求最高精度，而是作为一个结构清晰、可解释、容易扩展的基础 CNN baseline。它可以用于完成以下实验目的：

1. 验证 CNN 能够在 STL-10 上有效学习图像特征
2. 为后续数据增强、Dropout、BatchNorm、池化方式等实验提供基准
3. 为 Grad-CAM 可视化提供可解释的卷积特征层
4. 与更复杂模型，如 ResNet、VGG 风格网络进行对比

从目前训练日志看，BasicCNN 的训练 loss 和验证 loss 均呈下降趋势，验证准确率逐步提升到约 60%，说明该网络已经有效学习到 STL-10 的类别特征，并且没有出现明显过拟合。因此它可以作为课程项目中的基础模型结果。

8. 可改进方向

如果希望进一步提升性能，可以从以下方向扩展：

1. 增加训练轮数，例如训练 40 到 60 个 epoch
2. 增大网络宽度，例如通道数改为 64, 128, 256, 512
3. 加入更多数据增强，例如 ColorJitter、RandomRotation
4. 尝试不同池化方式，例如 Average Pooling
5. 对比不同激活函数，例如 ReLU、Sigmoid、Tanh
6. 使用 ResNet18 等成熟 CNN 结构进行对比实验
7. 加入 Grad-CAM 可视化，分析模型关注区域

这些方向也正好对应课程项目中关于模型结构、数据增强、正则化和可解释性的实验要求。